МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп'ютерних технологій
Кафедра системного проектування

ЗВІТ

про виконання лабораторної роботи №2
з курсу «Професійний Python»

«Структури даних та функціональна декомпозиція»

Виконав:
студент 3 курсу, групи ФЕП-31с
Гарбіч Данило

Перевірив:
професор
Свелеба Сергій Андрійович

Львів — 2026

2. Номер і тема лабораторної роботи

Лабораторна робота №2. «Структури даних та функціональна декомпозиція».

3. Мета роботи

Сформувати практичні навички професійного використання базових (`list`, `tuple`, `set`, `dict`) та розширених (`collections.Counter`, `collections.defaultdict`, `collections.deque`) структур даних Python, навчитися обирати оптимальну структуру відповідно до поставленої інженерної задачі, оцінювати асимптотичну часову складність основних операцій за допомогою нотації Big-O та проводити функціональну декомпозицію програми на невеликі функції з чітко визначеною відповідальністю. Опанувати роботу зі змінною кількістю аргументів (`*args`, `**kwargs`), анонімними функціями (`lambda`), замиканнями (`closures`) та декораторами (`decorators`).

4. Номер індивідуального варіанта

Варіант №9: «Облік спортивних результатів».

5. Постановка задачі

Створити модуль обробки структурованих даних для варіанта №9 («Облік спортивних результатів»).

Вимоги до функціоналу:

- прийом та збереження набору структурованих записів спортсменів (ID, ім'я, вид спорту, результат, категорія, кортеж незмінних метаданих);
- використання списку (`list`), кортежу (`tuple`), множини (`set`) та словника (`dict`);
- застосування спеціалізованих колекцій: `Counter` (підрахунок), `defaultdict` (одно- та багаторівневе групування), `deque` (кільцевий буфер історії операцій);
- реалізація спискових включень (`list/set/dict comprehensions`) та генераторних виразів;
- декомпозиція коду на спеціалізовані функції з використанням `*args`, `**kwargs` та `lambda`;
- реалізація фабрики замикань (`closure`) для гнучкої фільтрації даних;
- створення декоратора аудиту часу (`@measure_time`) та параметризованого декоратора (`@repeat`);
- проведення теоретичного аналізу складності операцій та експериментального бенчмарку (List vs Dict) на наборах 1 000, 10 000, 100 000 записів.

6. Опис структури даних

У проєкті використано модель структурованих спортивних результатів на базі словника із вкладеним кортежем метаданих:

```
athlete = {
    "id": 1,
    "name": "Ярослава Магучіх",
    "sport": "Стрибки у висоту",
    "result": 2.10,
    "category": "Дорослі",
    "meta": ("WR", "Україна", 2024),  # tuple: тип рекорду, країна, рік
}
```

7. Пояснення вибору list, tuple, set, dict

- **`list`**: обрано для збереження головного впорядкованого динамічного реєстру спортсменів. Забезпечує додавання за амортизований час O(1), збереження черговості та гнучке сортування за допомогою Timsort за час O(n log n).
- **`tuple`**: обрано для незмінних метаданих рекорду (тип досягнення, країна, рік). Гарантує цілісність інформації від випадкових змін, займає менше оперативної пам'яті, ніж список.
- **`set`**: обрано для формування множин унікальних дисциплін і категорій. Дозволяє миттєво відсікати дублікати та виконувати перевірку належності за час O(1), а також підтримує теоретико-множинні операції (перетин &).
- **`dict`**: обрано як для представлення окремого об'єкта спортсмена з іменованими полями, так і для побудови вторинних індексів за ID. Завдяки реалізації на хеш-таблицях забезпечує швидкий доступ O(1).

8. Приклад використання collections

У модулях проєкту застосовано три структури модуля `collections`:

1. `collections.Counter` — підрахунок кількості спортсменів у дисциплінах:

   ```
   sport_counts = Counter(item["sport"] for item in records)
   ```

2. `collections.defaultdict` — групування спортсменів за категоріями та багаторівневе групування (вид спорту -> категорія -> список):

   ```
   nested = defaultdict(lambda: defaultdict(list))
   ```

3. `collections.deque` — кільцевий буфер фіксованої довжини (`maxlen=20`) для журналу останніх операцій у системі:

   ```
   operation_history = deque(maxlen=20)
   ```

9. Структура проєкту

Проєкт реалізовано за архітектурним принципом `src-layout`:

```
lab2/
├── .gitignore                      # Ігнорування кешу, .venv та службових файлів
├── pyproject.toml                  # Стандартизовані метадані пакету (PEP 621)
├── README.md                       # Технічна документація проєкту
├── REPORT.md                       # Текст звіту до лабораторної роботи
│
├── src/
│   └── data_processor/             # Пакет обробки даних
│       ├── __init__.py             # Версія програми (__version__ = "0.1.0")
│       ├── data.py                 # Набори даних (list of dicts, tuples, sets)
│       ├── decorators.py           # Декоратори @measure_time, @repeat, deque
│       ├── processors.py           # Фільтрація, індексація, групування, Counter
│       ├── analytics.py            # Агрегація, сортування, closures, *args, **kwargs
│       └── main.py                 # Точка входу, виклики та бенчмарк
│
└── tests/
    └── test_data_processor.py      # Модульні тести алгоритмів
```

10. Основні функції програми

Функція	Модуль	Призначення
`get_unique_sports`	`processors.py`	Отримання множини унікальних дисциплін через set comprehension
`create_id_index`	`processors.py`	Побудова хеш-індексу ID -> Athlete через dict comprehension
`filter_by_sport`	`processors.py`	Фільтрація спортсменів за видом спорту через list comprehension
`group_by_sport`	`processors.py`	Групування записів за видом спорту через `defaultdict(list)`
`count_by_sport`	`processors.py`	Підрахунок частот появи дисциплін через `Counter`
`nested_grouping`	`processors.py`	Дворівневе групування (вид спорту -> категорія)
`calculate_average`	`analytics.py`	Обчислення середнього бала через `generator expression` під `@measure_time`
`find_best_athlete`	`analytics.py`	Пошук рекордсмена за допомогою `max()` та `lambda`
`sort_by_result`	`analytics.py`	Сортування результатів за спаданням через `sorted()` та `lambda`
`calculate_average_values`	`analytics.py`	Розрахунок середнього з довільної кількості чисел через `*args`
`create_record`	`analytics.py`	Динамічне створення запису через `**kwargs`
`create_category_filter`	`analytics.py`	Фабрика замикань (`closure`) для фільтрації за категорією
`pipeline_processing`	`analytics.py`	Послідовна конвеєрна обробка даних через ланцюг перетворень
`measure_time`	`decorators.py`	Декоратор вимірювання часу виконання функцій
`repeat`	`decorators.py`	Параметризований декоратор повторного виклику функцій

11. Приклад comprehensions

```
# 1. List comprehension
jumpers = [item for item in records if item["sport"].lower() == "стрибки у висоту"]

# 2. Set comprehension
unique_sports = {item["sport"] for item in records}

# 3. Dict comprehension
id_index = {item["id"]: item for item in records}

# 4. Generator expression (без створення проміжного списку)
total_score = sum(item["result"] for item in records)
```

12. Приклад *args

```
def calculate_average_values(*values: float) -> float:
    if not values:
        return 0.0
    return round(sum(values) / len(values), 2)

# Виклик:
result = calculate_average_values(12.5, 14.0, 18.2, 20.1)  # 16.20
```

13. Приклад **kwargs

```
def create_record(**fields: Any) -> dict[str, Any]:
    return dict(fields)

# Виклик:
new_record = create_record(
    id=9,
    name="Жан Беленюк",
    sport="Греко-римська боротьба",
    result=1.0,
    category="Майстри",
    meta=("OG-Gold", "Україна", 2021)
)
```

14. Приклад lambda

```
# Пошук спортсмена з максимальним результатом:
best = max(records, key=lambda item: item["result"])

# Сортування рейтингу за спаданням результату:
ranking = sorted(records, key=lambda item: item["result"], reverse=True)
```

15. Пояснення closure (замикання)

Замикання (closure) — це внутрішня функція, яка запам'ятовує змінні зі свого лексичного контексту (зовнішньої функції) навіть після того, як зовнішня функція завершила своє виконання і була вилучена зі стека викликів.

```
def create_category_filter(category: str) -> Callable[[dict[str, Any]], bool]:
    target = category.strip().lower()  # Змінна зберігається в замиканні

    def predicate(athlete: dict[str, Any]) -> bool:
        return athlete["category"].strip().lower() == target
    return predicate

# Використання:
filter_senior = create_category_filter("Дорослі")
seniors = [a for a in athletes if filter_senior(a)]
```

Функція `create_category_filter` завершила роботу, однак згенерована функція `predicate` продовжує утримувати посилання на змінну `target` у своїй внутрішній комірці пам'яті `__closure__`.

16. Пояснення decorator (декоратор)

Декоратор — це шаблон проектування та синтаксична конструкція Python, що дозволяє динамічно розширювати функціональність існуючої функції без зміни її вихідного коду.

```
def measure_time(func: Callable[..., Any]) -> Callable[..., Any]:
    @wraps(func)
    def wrapper(*args: Any, **kwargs: Any) -> Any:
        start_time = perf_counter()
        result = func(*args, **kwargs)
        elapsed = perf_counter() - start_time
        print(f"[TIMING] {func.__name__}: {elapsed:.8f} s")
        operation_history.append(f"{func.__name__} executed in {elapsed:.8f} s")
        return result
    return wrapper
```

Декоратор приймає функцію `func`, обертає її у внутрішню функцію `wrapper` із заміром часу та реєстрацією в черзі `deque`, зберігає метадані функції через `@wraps` та прозоро повертає результат виклику.

17. Результат роботи програми

```
======================================================================
  ЛАБОРАТОРНА РОБОТА №2: СТРУКТУРИ ДАНИХ ТА ФУНКЦІОНАЛЬНА ДЕКОМПОЗИЦІЯ
  Варіант 9: Облік спортивних результатів
======================================================================

=== Початковий список спортсменів (list of dicts з tuple та set) ===
-------------------------------------------------------------------------------------------------------
ID   | Ім'я                  | Вид спорту            | Результат  | Категорія    | Мета
-------------------------------------------------------------------------------------------------------
1    | Ярослава Магучіх      | Стрибки у висоту      | 2.10       | Дорослі      | ('WR', 'Україна
2    | Ірина Геращенко       | Стрибки у висоту      | 2.00       | Дорослі      | ('PB', 'Україна
3    | Юлія Левченко         | Стрибки у висоту      | 1.95       | Дорослі      | ('SB', 'Україна
4    | Олег Верняєв          | Спортивна гімнастика  | 15.30      | Майстри      | ('OG', 'Україна
5    | Ілля Ковтун           | Спортивна гімнастика  | 15.65      | Юніори       | ('ECH', 'Україн
6    | Михайло Романчук      | Плавання              | 880.20     | Дорослі      | ('NR', 'Україна
7    | Олександр Желтяков    | Плавання              | 895.10     | Юніори       | ('WJC', 'Україн
8    | Марина Бех-Романчук   | Стрибки у довжину     | 6.93       | Дорослі      | ('PB', 'Україна
-------------------------------------------------------------------------------------------------------

[SET] Унікальні види спорту (4): {'Спортивна гімнастика', 'Плавання', 'Стрибки у довжину', 'Стрибки
[SET] Унікальні категорії (3): {'Юніори', 'Майстри', 'Дорослі'}
[SET] Операція перетину з еталонними категоріями: {'Юніори', 'Майстри', 'Дорослі'}

[DICT INDEX] Швидкий пошук за ID=5 (O(1)): {'id': 5, 'name': 'Ілля Ковтун', 'sport': 'Спортивна гімн

=== Спортсмени дисципліни 'Стрибки у висоту' ===
-------------------------------------------------------------------------------------------------------
ID   | Ім'я                  | Вид спорту            | Результат  | Категорія    | Мета
-------------------------------------------------------------------------------------------------------
1    | Ярослава Магучіх      | Стрибки у висоту      | 2.10       | Дорослі      | ('WR', 'Україна
2    | Ірина Геращенко       | Стрибки у висоту      | 2.00       | Дорослі      | ('PB', 'Україна
3    | Юлія Левченко         | Стрибки у висоту      | 1.95       | Дорослі      | ('SB', 'Україна
-------------------------------------------------------------------------------------------------------

[GROUPING (defaultdict)] Групування спортсменів за видами спорту:
  * Стрибки у висоту (3): Ярослава Магучіх, Ірина Геращенко, Юлія Левченко
  * Спортивна гімнастика (2): Олег Верняєв, Ілля Ковтун
  * Плавання (2): Михайло Романчук, Олександр Желтяков
  * Стрибки у довжину (1): Марина Бех-Романчук

[COLLECTIONS.COUNTER] Кількість спортсменів за видами спорту:
  * Стрибки у висоту: 3
  * Спортивна гімнастика: 2
  * Плавання: 2
  * Стрибки у довжину: 1

[TIMING] calculate_average: 0.00000670 s
[AGGREGATION] Середній результат усіх учасників: 227.40

[LAMBDA MAX] Спортсмен з абсолютним максимумом: Олександр Желтяков (895.10)

=== Рейтинг спортсменів за результатом (sorted + lambda) ===
-------------------------------------------------------------------------------------------------------
ID   | Ім'я                  | Вид спорту            | Результат  | Категорія    | Мета
-------------------------------------------------------------------------------------------------------
7    | Олександр Желтяков    | Плавання              | 895.10     | Юніори       | ('WJC', 'Україн
6    | Михайло Романчук      | Плавання              | 880.20     | Дорослі      | ('NR', 'Україна
5    | Ілля Ковтун           | Спортивна гімнастика  | 15.65      | Юніори       | ('ECH', 'Україн
4    | Олег Верняєв          | Спортивна гімнастика  | 15.30      | Майстри      | ('OG', 'Україна
8    | Марина Бех-Романчук   | Стрибки у довжину     | 6.93       | Дорослі      | ('PB', 'Україна
1    | Ярослава Магучіх      | Стрибки у висоту      | 2.10       | Дорослі      | ('WR', 'Україна
```

```
2    | Ірина Геращенко        | Стрибки у висоту       | 2.00       | Дорослі      | ('PB', 'Україна
3    | Юлія Левченко          | Стрибки у висоту       | 1.95       | Дорослі      | ('SB', 'Україна
-------------------------------------------------------------------------------------------------------

=== Фільтрація через closure (Категорія: 'Дорослі') ===
-------------------------------------------------------------------------------------------------------
ID   | Ім'я                   | Вид спорту             | Результат  | Категорія    | Мета
-------------------------------------------------------------------------------------------------------
1    | Ярослава Магучіх       | Стрибки у висоту       | 2.10       | Дорослі      | ('WR', 'Україна
2    | Ірина Геращенко        | Стрибки у висоту       | 2.00       | Дорослі      | ('PB', 'Україна
3    | Юлія Левченко          | Стрибки у висоту       | 1.95       | Дорослі      | ('SB', 'Україна
6    | Михайло Романчук       | Плавання               | 880.20     | Дорослі      | ('NR', 'Україна
8    | Марина Бех-Романчук    | Стрибки у довжину      | 6.93       | Дорослі      | ('PB', 'Україна
-------------------------------------------------------------------------------------------------------

[*ARGS] Розрахунок середнього з довільної кількості аргументів: 16.20
[**KWARGS] Створення нового запису спортсмена: {'id': 9, 'name': 'Жан Беленюк', 'sport': 'Греко-римс

[PARAMETERIZED DECORATOR] Демонстрація @repeat(count=2):
  -> Виконання повторюваної дії через @repeat(2)
  -> Виконання повторюваної дії через @repeat(2)

[ADVANCED] Вкладене групування (Спорт -> Категорія -> Кількість):
  * Стрибки у висоту -> Дорослі: 3 чол.
  * Спортивна гімнастика -> Майстри: 1 чол.
  * Спортивна гімнастика -> Юніори: 1 чол.
  * Плавання -> Дорослі: 1 чол.
  * Плавання -> Юніори: 1 чол.
  * Стрибки у довжину -> Дорослі: 1 чол.

[ADVANCED] Конвеєрна обробка (Pipeline): Фільтр юніорів -> Сортування за результатом

=== Результат конвеєра (Pipeline) ===
-------------------------------------------------------------------------------------------------------
ID   | Ім'я                   | Вид спорту             | Результат  | Категорія    | Мета
-------------------------------------------------------------------------------------------------------
7    | Олександр Желтяков     | Плавання               | 895.10     | Юніори       | ('WJC', 'Україн
5    | Ілля Ковтун            | Спортивна гімнастика   | 15.65      | Юніори       | ('ECH', 'Україн
-------------------------------------------------------------------------------------------------------

[COLLECTIONS.DEQUE] Останні операції у системі:
  - calculate_average executed in 0.00000670 s

--- ЕКСПЕРИМЕНТАЛЬНА ЧАСТИНА: БЕНЧМАРК (List Search vs Dict Search) ---
Кількість записів:    1000 | List: 0.00002960 s | Dict: 0.00000040 s | Прискорення: 74.0x
Кількість записів:   10000 | List: 0.00028760 s | Dict: 0.00000110 s | Прискорення: 261.5x
Кількість записів:  100000 | List: 0.00314190 s | Dict: 0.00000790 s | Прискорення: 397.7x
```

18. Таблиця складності operations (Big-O)

Операція	Структура даних	Середня складність	Найгірша складність	Пояснення
Пошук за ID перебором	`list`	O(n)	O(n)	Послідовний лінійний перебір елементів до першого збігу
Пошук за ID за індексом	`dict`	O(1)	O(n)	Прямий доступ за хешем ключа в асоціативній таблиці
Перевірка наявності	`set`	O(1)	O(n)	Хеш-пошук у множині унікальних ключів
Сортування за результатом	`list`	O(n log n)	O(n log n)	Гібридний алгоритм сортування Timsort у мові Python
Додавання в кінець (append)	`list`	O(1)	O(n)	Амортизована константа (перевиділення пам'яті лише зрідка)
Додавання/ вилучення з країв	`deque`	O(1)	O(1)	Швидка робота завдяки двозв'язній блочній структурі

19. Результати benchmark

Кількість записів (n)	Час пошуку в list (O(n))	Час пошуку в dict (O(1))	Фактичне прискорення
1 000	0.00002960 с	0.00000040 с	**74.0**×
10 000	0.00028760 с	0.00000110 с	**261.5**×
100 000	0.00314190 с	0.00000790 с	**397.7**×

Висновок за результатами експерименту: зі зростанням обсягу вибірки час пошуку у списку зростає прямо пропорційно кількості елементів (строга лінійна залежність $O(n)$). Натомість пошук у словнику залишається в межах часток мікросекунди ($O(1)$). На обсязі в 100 000 записів пошук за хеш-індексом виявився швидшим у **397.7 раза**, що повністю підтверджує теоретичні викладки.

20. Відповіді на контрольні запитання

1. Що таке list?

list — це впорядкована змінювана (mutable) динамічна послідовність довільних елементів з доступом за індексом.

2. Чим list відрізняється від tuple?

list є змінюваним (mutable), а tuple — незмінним (immutable). Кортежі займають менше пам'яті, працюють швидше та можуть бути ключами словника.

3. Чому tuple називається immutable?

Тому що після створення об'єкта tuple неможливо змінити його довжину або замінити посилання на елементи.

4. Для чого використовується set?

set використовується для збереження невпорядкованої колекції унікальних елементів, швидкої перевірки входження O(1) та математичних операцій над множинами.

5. Яка властивість елементів set є важливою?

Всі елементи множини повинні бути обов'язково хешованими (hashable) — реалізовувати незмінний метод __hash__() та метод рівності __eq__().

6. Що таке dict?

dict — це асоціативна структура даних, яка зберігає пари ключ-значення з високою швидкістю доступу за ключем.

7. Що таке key-value structure?

Це принцип зберігання даних, де доступ до кожного значення (value) здійснюється за його унікальним ідентифікатором — ключем (key).

8. Чому пошук key у dict зазвичай швидкий?

Оскільки dict побудовано на базі хеш-таблиці: індекс комірки обчислюється напряму через hash(key), забезпечуючи константний час O(1).

9. Що таке hashability?

Хешованість — це здатність об'єкта повертати незмінне числове значення хешу через функцію hash() протягом усього свого життєвого циклу.

10. Чому list не може бути key у dict?

Оскільки list є змінюваним (mutable) і не має методу __hash__(); зміна списку зробила б неможливим пошук елемента в хеш-таблиці.

11. Для чого використовується Counter?

collections.Counter призначений для автоматичного підрахунку кількості появ повторюваних елементів у послідовностях.

12. Для чого використовується defaultdict?

collections.defaultdict автоматично викликає функцію-фабрику для генерації значення за замовчуванням при зверненні до неіснуючого ключа.

13. Для чого використовується deque?

collections.deque (двозбічна черга) призначена для швидкого додавання та видалення елементів з обох кінців за гарантований час O(1).

14. Що таке list comprehension?

Лаконічний синтаксис для створення нових списків шляхом ітерації та фільтрації: [expr for item in iterable if cond].

15. Що таке set comprehension?

Синтаксис для створення множин унікальних елементів: {expr for item in iterable if cond}.

16. Що таке dict comprehension?

Синтаксис для створення словника пар ключ-значення: {k_expr: v_expr for item in iterable if cond}.

17. Що таке generator expression?

Лінивий генератор (expr for item in iterable), що генерує елементи по одному без завантаження всього набору в оперативну пам'ять.

18. Що означає O(1)?

Константна складність — час виконання операції не залежить від розміру вхідних даних n.

19. Що означає O(n)?

Лінійна складність — час виконання операції зростає прямо пропорційно до кількості елементів n.

20. Що означає O(n log n)?

Лінійно-логарифмічна складність — характерна для швидких алгоритмів сортування (наприклад, Timsort).

21. Що означає $O(n^2)$?

Квадратична складність — час виконання зростає пропорційно квадрату обсягу даних (характерно для подвійних вкладених циклів).

22. Яка приблизна складність пошуку у list?

O(n) для несортованого списку, оскільки необхідно послідовно перевірити кожен елемент.

23. Яка середня складність пошуку у set?

O(1) у середньому завдяки прямому доступу через хеш-таблицю.

24. Яка середня складність пошуку key у dict?

O(1) у середньому завдяки адресації за хешем ключа.

25. Чому правильна структура даних може прискорити програму?

Зниження асимптотичної складності (наприклад, з O(n) на O(1)) кардинально зменшує кількість процесорних операцій на великих обсягах даних.

26. Що таке функціональна декомпозиція?

Розбиття складної програми на невеликі, логічно незалежні функції, кожна з яких виконує рівно одну задачу.

27. Які переваги малих functions?

Висока читабельність, простота покриття автоматичними юніт-тестами, легкість налагодження та можливість повторного використання.

28. Що таке positional argument?

Аргумент, значення якого передається у функцію згідно з його позиційним порядком у сигнатурі.

29. Що таке keyword argument?

Іменований аргумент, що передається у форматі ім'я=значення, що дозволяє передавати параметри в довільному порядку.

30. Для чого використовується *args?

Для прийому довільної кількості позиційних аргументів, які всередині функції пакуються в кортеж tuple.

31. Для чого використовується **kwargs?

Для прийому довільної кількості іменованих аргументів, які всередині функції пакуються в словник dict.

32. Що таке lambda function?

Коротка анонімна однорядкова функція: lambda args: expr.

33. Де доцільно використовувати lambda?

У функціях вищого порядку (як ключ сортування в sorted/max/min або простий предикат фільтрації).

34. Чому складну business logic не варто записувати в lambda?

Lambda обмежена одним виразом, не має зрозумілого імені для налагодження та погіршує читабельність коду.

35. Що таке closure?

Внутрішня функція, яка запам'ятовує змінні зовнішньої функції навіть після завершення роботи зовнішньої функції.

36. Які variables може пам'ятати closure?

Будь-які нелокальні змінні з області видимості зовнішньої функції, до яких звертається внутрішня функція.

37. Що таке decorator?

Функція вищого порядку, яка приймає іншу функцію та розширює її поведінку без зміни її вихідного коду.

38. Для чого застосовуються decorators?

Для профілювання часу виконання, логування, кешування результатів, аутентифікації та валідації вхідних даних.

39. Для чого використовується functools.wraps?

Для збереження імені (__name__), документації (__doc__) та анотацій оригінальної функції у функції-обгортці.

40. Чому потрібно оцінювати складність алгоритму?

Для прогнозування поведінки програми під навантаженням та запобігання зависанням і перевитратам пам'яті на великих даних.

21. Висновок

Під час виконання лабораторної роботи №2 було детально досліджено та практично реалізовано ключові принципи професійної роботи зі структурами даних та функціональної декомпозиції в Python:

1. **Структури даних:** реалізовано комбіновану обробку даних за допомогою `list`, `tuple`, `set` та `dict` відповідно до індивідуального варіанта № 9 («Облік спортивних результатів»).
2. **Модуль collections:** використано `Counter` для аналітики розподілу за видами спорту, `defaultdict` для багаторівневого групування та `deque` для ведення історії операцій.
3. **Функціональні засоби:** декомпозовано систему на невеликі функції, впроваджено `*args`, `**kwargs`, `lambda`, замикання (`closures`), конвеєрну обробку (`pipeline`) та декоратори (`@measure_time`, `@repeat`).
4. **Оцінка складності та бенчмарк:** підтверджено теоретичну складність операцій на практиці. Пошук через хеш-індекс `dict` ($O(1)$) продемонстрував перевагу над лінійним перебором у `list` ($O(n)$) до **397.7 раза** на вибірці у 100 000 елементів.

Створений комплекс є модульним, повністю протестованим та готовим до використання у наступних роботах курсу.